

While the above skeleton code depicts a very simple network consisting of a single stage convolution and pooling, followed by a single stage feed-forward neural network, this can be extended to multiple stages of convolution/pooling and a multi-layer feed-forward neural network connected at the end for classification. It is also possible to use a combination of a recurrent neural network followed by a CNN. For instance, the input sentence can be passed through a gated recurrent neural network layer to obtain a sentence representation, which is then passed through multiple stages of convolution and pooling with the output layer being a multi-layer perceptron classifier.

There have been a number of papers written on how to use neural networks for hate speech detection. A good survey on the topic can be found in the paper at <http://www.aclweb.org/anthology/W17-1101>. A technique that combines GRUs with convolutional neural networks is described in the paper by Zhang available at https://2018.eswc-conferences.org/wp-content/uploads/2018/02/ESWC2018_paper_48.pdf. A number of data sets are available for hate speech detection, which are described in the above paper and are also available for download at <https://github.com/ziqizhang/chase>. A Kaggle competition data set on classifying toxic comments is also available at <https://www.kaggle.com/c/jigsaw-toxic-comment->

[classification-challenge/data](#). Readers can experiment with different models on these data sets and compare their results with the published results in these papers.

All these are supervised learning techniques and hence require huge volumes of labelled data. As we know, getting labelled data is cumbersome, error-prone and expensive, especially given the subjective nature of this task. In next month's column, we will discuss some of the unsupervised approaches to hate speech classification.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Here's wishing all our readers happy coding until next month! **END** 🐧

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working as a research scientist in Conduent Labs India (formerly Xerox India Research Centre). Her interests include compilers, programming languages, file systems and natural language processing. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' at <http://www.linkedin.com/groups?home=&gid=2339182>.

Continued from page 43...

The producer will wait on the input from *stdin* and publish it to the Kafka cluster. By default, every new line is published as a new message, and the default producer's properties are specified in the *config/producer.properties* file.

The output is:

```
$ bin/kafka-console-producer.sh --broker-list localhost:9092
--topic Hello-Kafka
[2018-05-31 16:53:30,931]
Hello
The first message
```

c) Starting consumers to receive messages

Similar to producer, the default consumer properties are specified in the *config/consumer.properties* file. Open a new terminal and type the following example for consuming messages (*Hello-Kafka* is the topic already created). Here is an example:

```
bin/kafka-console-consumer.sh --zookeeper localhost:2181 --
topic Hello-Kafka
--from-beginning
Output
Hello
The first message
```

We are now able to enter messages from the producer's

terminal and see them appearing in the consumer's terminal. So is the case with single node multiple brokers. Since we require multiple broker instances, copy the existing *server.properties* file into two new *config* files and rename them accordingly. Next, edit *broker.id*, the port of server socket and *log.dirs*. Start multiple brokers, create topics, start producers and consumers, and the cycle continues.

To recapitulate, Kafka supports many industrial applications such as Twitter, LinkedIn, Oracle, Mozilla, Netflix, etc. Kafka gets used for stream processing, to track website activity, for metrics collection and monitoring, log aggregation, real-time analytics, ingesting data into Spark and Hadoop, CQRS, replay messages, error recovery, and so on. Being designed as a distributed, scalable infrastructure component makes it an ideal backbone through which services exchange and buffer events. This article will be useful in creating Kafka producers and consumers using Kafka clients which are available for most programming languages. **END** 🐧

References

- [1] https://www.tutorialspoint.com/apache_kafka/index.htm
- [2] <http://kafka.apache.org/intro>

By: Neethu C. Sekar

The author is an open source enthusiast, currently working as an assistant professor in the Department of Computer Science, AmalJyothi College of Engineering, Kerala. She can be reached at nitucskr@gmail.com.